Word vectors and recurrent networks

Class 4 of 24: Natural Language Processing

CSE 4811 / DS 4213 (Fall 2026)

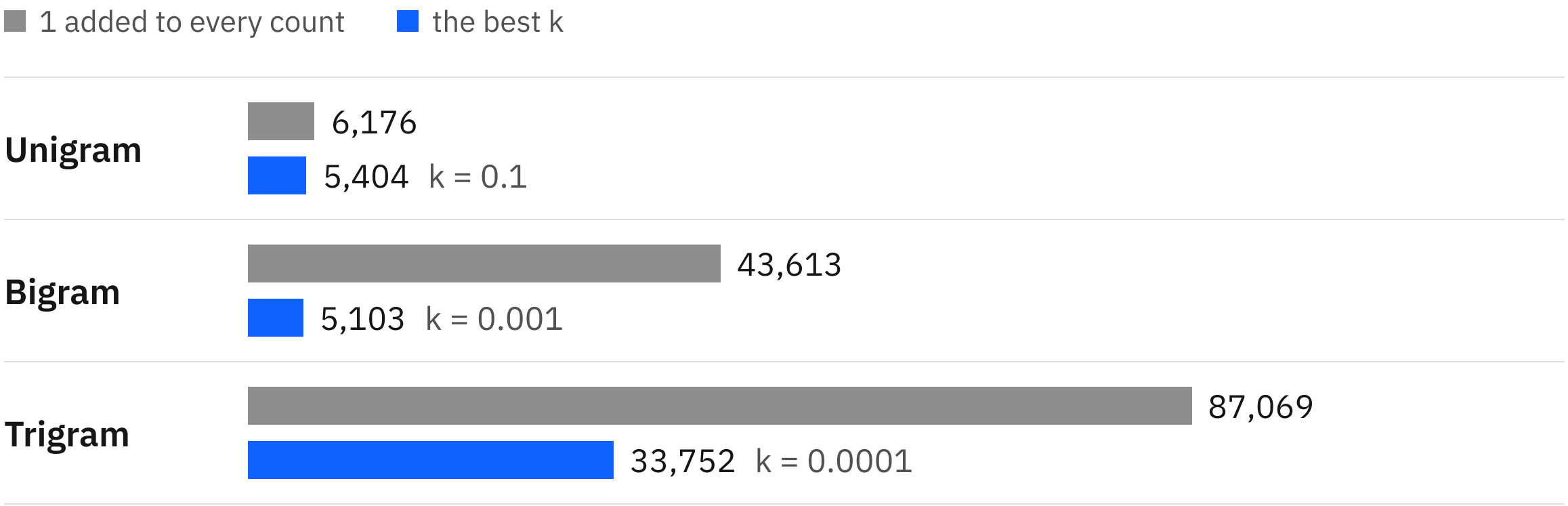
Muhammad Anwarul Azim
Lecturer, Dept. of CSE
United International University

Recap

The n-gram model	$p(w_i \mid w_{i-N+1} \dots w_{i-1}) = \dfrac{\text{count}(w_{i-N+1} \dots w_i)}{\text{count}(w_{i-N+1} \dots w_{i-1})}$
Generation	$p(\text{next word} \mid \text{history})$
Perplexity	$\text{perplexity} = e^{\,\text{loss}}$
Unseen n-grams	add-one, add-k, backoff

Add-one, add-k and backoff give every n-gram a probability above 0.

Perplexity on C1



Problem: even with the best k, the trigram model is far behind, because most of its trigrams were never seen.

What is missing?

Add-one	adds 1 to every count
Add-k	adds a small k, chosen on the dev set
Backoff	uses a shorter history when the longer one was never seen

`film` and `movie` two separate rows of counts

Can a model learn which words are alike?

Part 1: Words as vectors

Each word becomes a vector, learned from the loss.

What can counting not do?

Cells in the table	$N = 1$	$N = 2$	$N = 3$
	$110{,}608$	$110{,}609^2 \approx 1.2 \times 10^{10}$	$110{,}609^3 \approx 1.4 \times 10^{15}$

Unseen n-grams

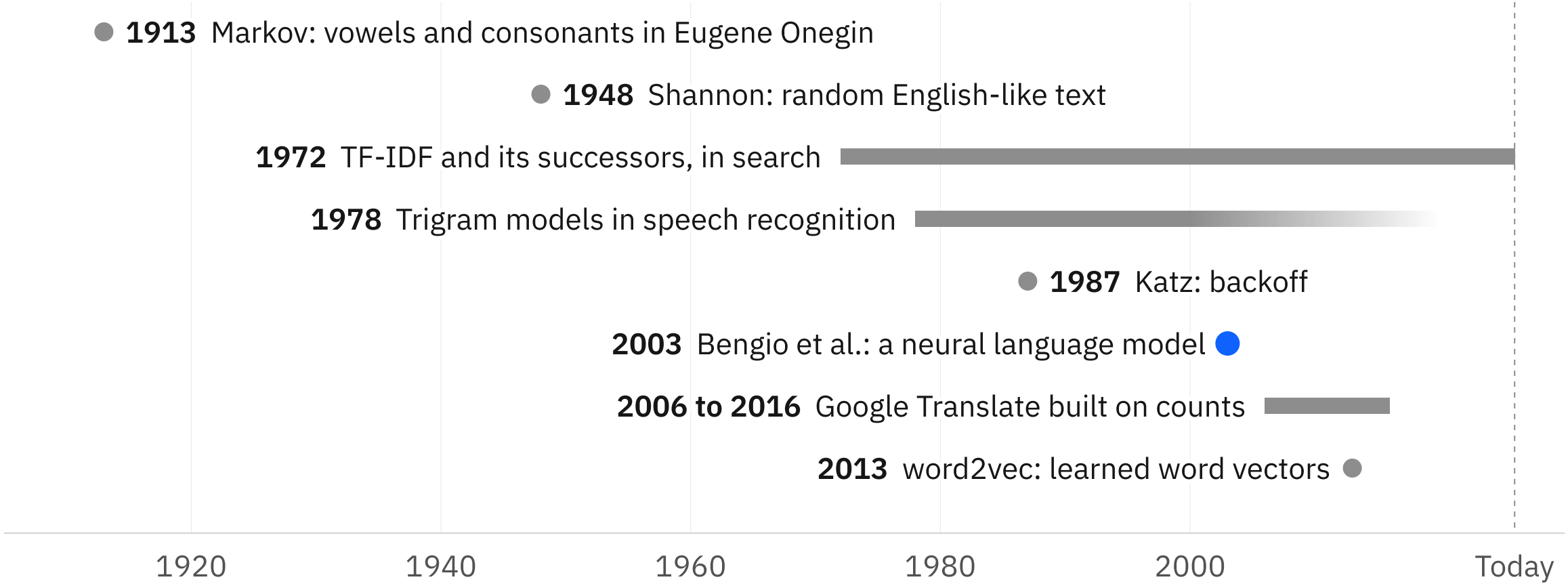
84.4% of the dev trigrams never occur in the training texts of their topic

Backoff

`panic` `among` `investors`

Problem: counting cannot use a long history, because most long n-grams never occur in the training texts.

How long did counting last?



So counting lasted about a hundred years, and search still uses it today.

Does the model know film and movie are alike?

Ten toy articles

```
match star film wins
star wins match
match star wins match
song star wins match
match wins match
```

```
wins match
star film song film wins
star film star song film wins
film star song star film
song star film match
```

Seven new articles

```
film star wins
star movie song movie wins
song star movie match
star movie song star movie
match star movie wins
song movie wins
star movie song film wins
```

	match	star	film	movie	wins	song	</s>	**Total**
film star	0	0	0	0	1	2	0	**3**
movie star	0	0	0	0	0	0	0	**0**

Test headline

Movie star wins

Which word comes after movie star?

The counts after movie star

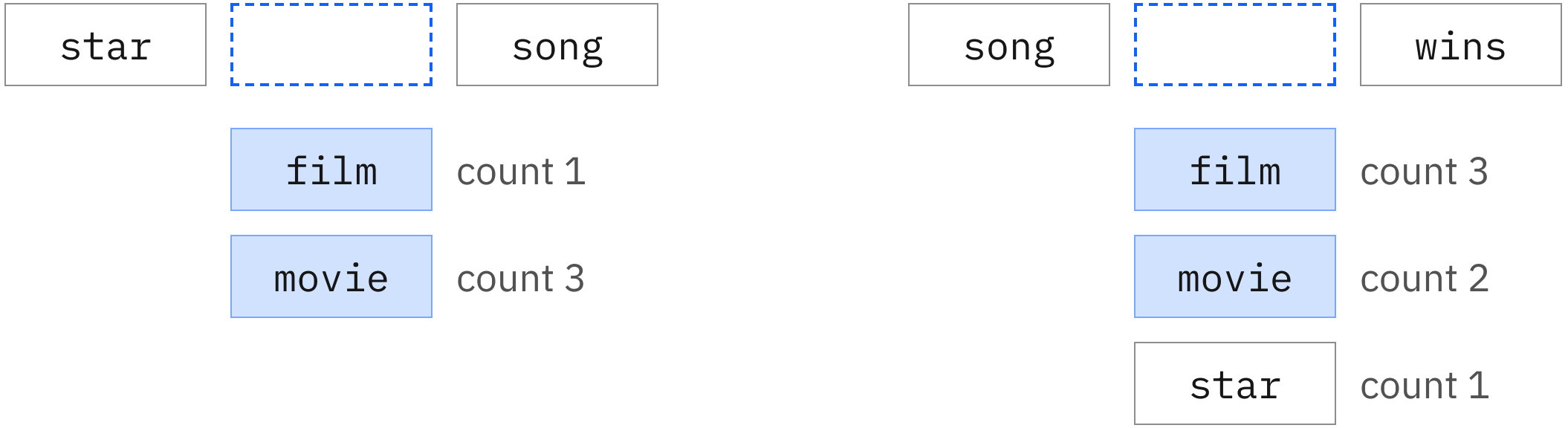
Test headline

`Movie star wins`

		match	star	film	movie	wins	song	</s>	**Total**	$p(\text{wins} \mid \text{history})$
Counts	film star	0	0	0	0	1	2	0	**3**	**1/3**
	movie star	0	0	0	0	0	0	0	**0**	**0/0**
Add-one	movie star	1	1	1	1	1	1	1	**7**	**1/7**
Backoff	movie star	0	0	5	6	4	2	0	**17**	**4/17**

Problem: what the counts learned about film star is never used for movie star, although film and movie are used alike.

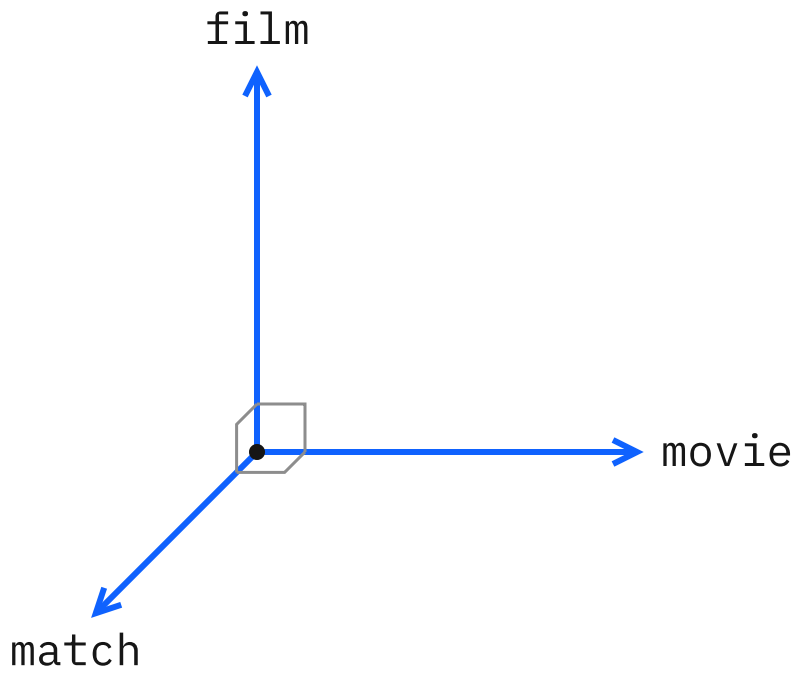
What makes two words alike?



Words that fit the same blanks are used alike, so we look at a word together with the words around it.

How is a word written now?

	match	star	film	movie	wins	song
film	0	0	1	0	0	0
movie	0	0	0	1	0	0
match	1	0	0	0	0	0

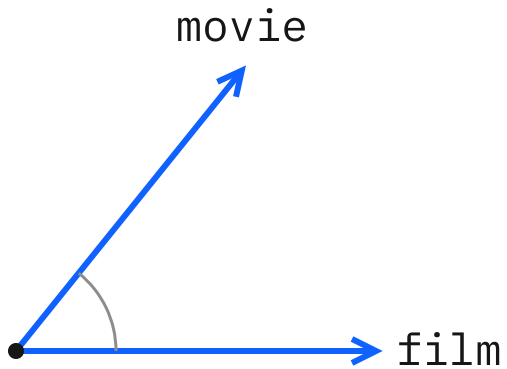


$$\cos(\text{film}, \text{movie}) = 0 \qquad \cos(\text{film}, \text{match}) = 0$$

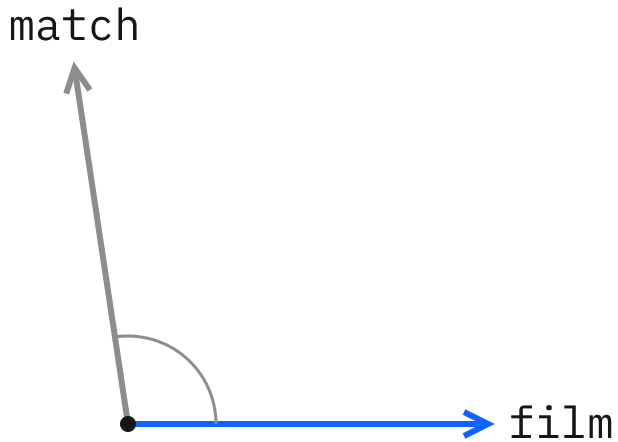
So film is as far from movie as it is from match.

A few numbers for each word

film	−1.21	−1.45	−2.53
movie	−2.20	0.62	−1.41
match	0.85	−1.69	0.96



$\cos(\text{film}, \text{movie}) = 0.63$



$\cos(\text{film}, \text{match}) = -0.15$

These numbers are not chosen by hand. They are learned from the 17 toy articles.

The paper

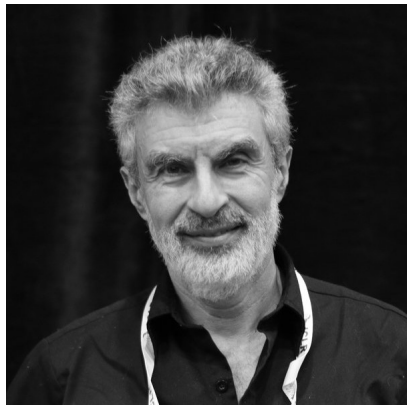
Journal of Machine Learning Research 3 (2003) 1137–1155 Submitted 4/02; Published 2/03

A Neural Probabilistic Language Model

Yoshua Bengio BENGIOY@IRO.UMONTREAL.CA
Réjean Ducharme DUCHARME@IRO.UMONTREAL.CA
Pascal Vincent VINCENTP@IRO.UMONTREAL.CA
Christian Jauvin JAUVINC@IRO.UMONTREAL.CA
Département d'Informatique et Recherche Opérationnelle
Centre de Recherche Mathématiques
Université de Montréal, Montréal, Québec, Canada

Three of the authors presented a shorter version in 2000 at NIPS, the conference now called NeurIPS.

Three researchers



Turing Award
2018

Yoshua Bengio

Université de Montréal and Mila

Neural language models and word vectors



Turing Award
2018

Nobel Prize
2024

Geoffrey Hinton

University of Toronto

Backpropagation, with Rumelhart and Williams, 1986



Turing Award
2018

Yann LeCun

New York University

Convolutional networks that read handwritten digits

In 2024 Hinton and John Hopfield shared the Nobel Prize in Physics, for discoveries that enable machine learning with neural networks.

Three ideas of the paper

A vector for each word `film` → −1.21 −1.45 −2.53

The next word from the vectors `movie` `star` → $p(\text{wins} \mid \text{movie star})$

Learned together Training text → Loss → Vectors and weights

Solved: words used alike can get nearby vectors, so what the model learns for film can also serve movie.

A vector for each word

E

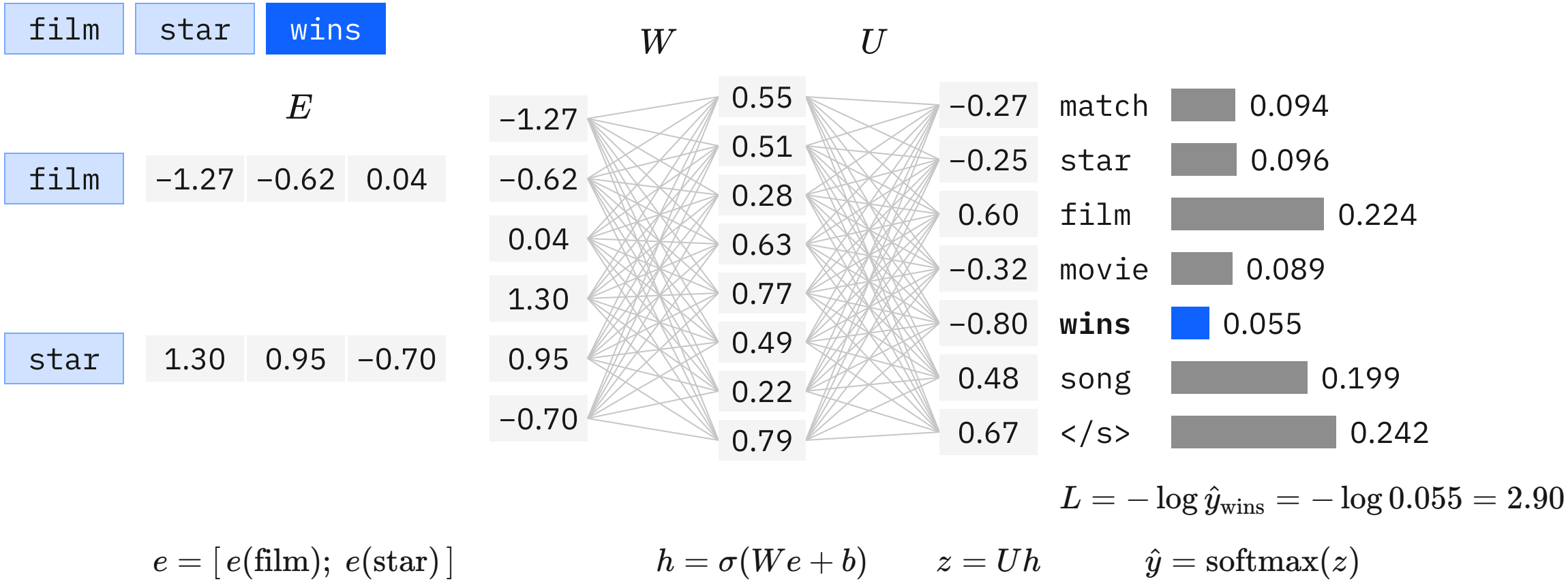
<s>	0.13	−0.13	0.64
match	0.10	−0.54	0.36
star	1.30	0.95	−0.70
film	−1.27	−0.62	0.04
movie	−2.33	−0.22	−1.25
wins	−0.73	−0.54	−0.32
song	0.41	1.04	−0.13

$$e(\text{film}) = [-1.27,\ -0.62,\ 0.04]$$



Training changes the numbers of E, so the points move.

The neural language model



The loss is $-\log \hat{y}$ for the true next word, wins: here 2.90.

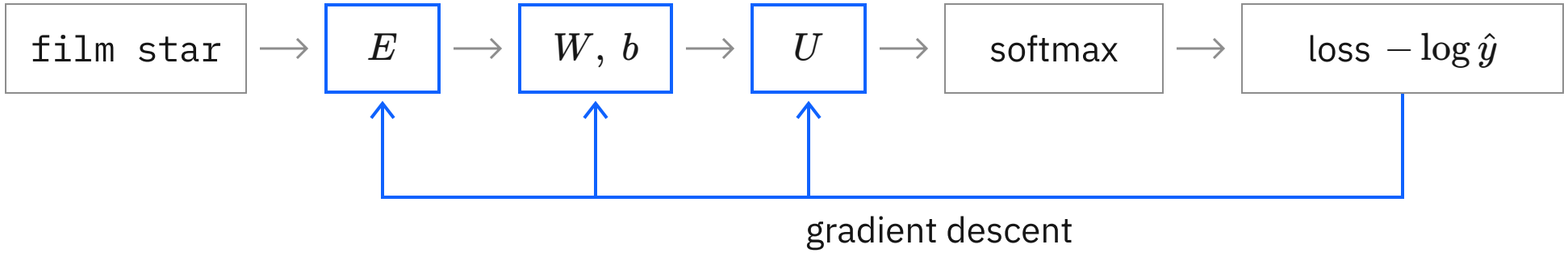
Training

Naive Bayes

count words → table $p(w \mid k)$ → loss $-\log p$

no way back: the loss cannot change the counts

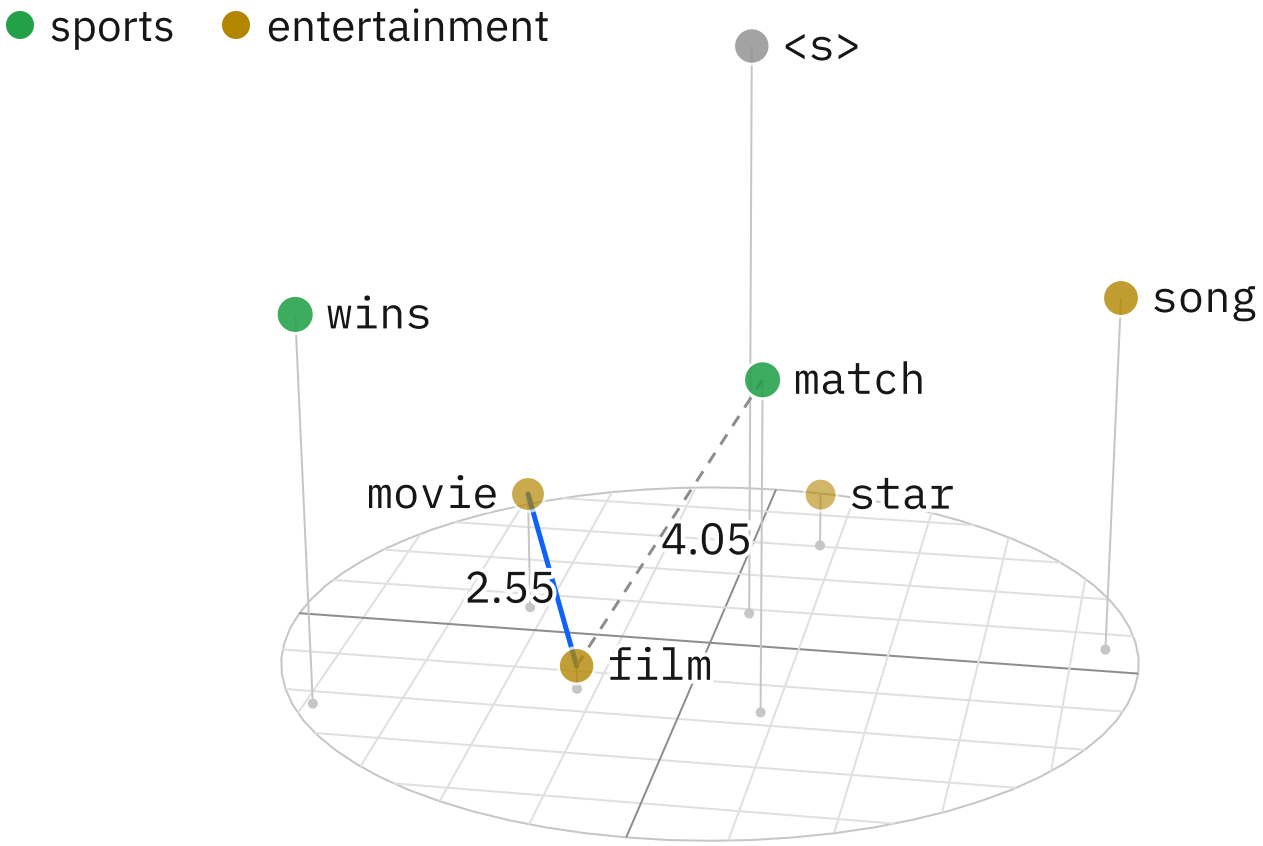
Neural language model

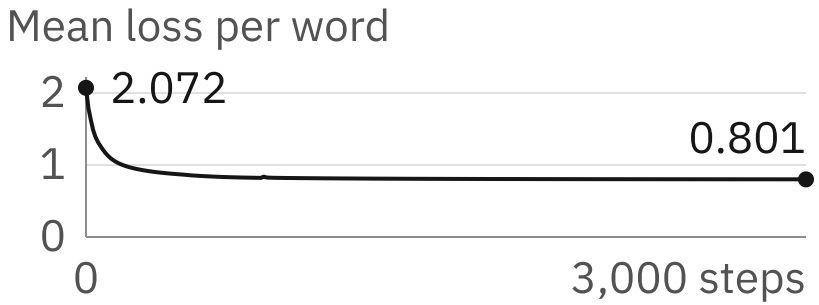


$\theta_{t+1} = \theta_t - \eta \nabla L$ for each of E, W, b and U

Solved: this model learns from its loss, which Naive Bayes could not do.

What is learned?





$p(\text{next word} \mid \text{history})$

	movie star	film star
match		
star	add-one: 1/7	
film		
movie		
wins	0.31	0.33
song	0.67	0.66
</s>		

Seen from every side, film and movie stay close, so what the model learns about film also applies to movie.

What is learned on C1?

Dev loss per word	All dev words 174,640 words	Unseen trigram 84.3% of the dev words	Seen trigram 15.7% of the dev words
Unigram	8.59	8.78	7.62
Bigram	8.54	9.41	3.89
Trigram	10.43	11.85	2.77
Neural	7.29	7.90	4.02

Nearest words by cosine

`presidential` national politics		`soccer` sports		`star` entertainment		`film` entertainment	
democratic	0.92	duke	0.90	oprah	0.85	music	0.87
republican	0.89	hockey	0.90	actor	0.82	celebrities	0.85
congressional	0.87	reds	0.89	craig	0.80	lee	0.83
th	0.87	basketball	0.89	bell	0.79	bluegrass	0.82
jewish	0.85	bowl	0.89	ross	0.79	media	0.82

Not every word gets neighbors this clear. For song and match, most of the five nearest words are unrelated.

The neural model writes

Topic

- business and economy
- entertainment
- international
- national politics
- science and technology
- sports

Neural language model

youth soccer coach to life tonight s children to death in ১১৭ old punter লাগল যদিও ও চন্ডিকা সমস্ত গত দলের অধিনায়ক বাবর পরে ও প্যাকেটবন্দি করেছে অস্ট্রেলিয়া মিশন ৬৩ রানে হারায় ঢাকা মোকাবেলায় ব্যর্থ ৯০ হয়েছিল ক্যারিয়ারের রেনের ছিল আরও </s>

Its word order is less fluent than the trigram model's. The trigram model follows only trigrams seen in training.

Counting and learning

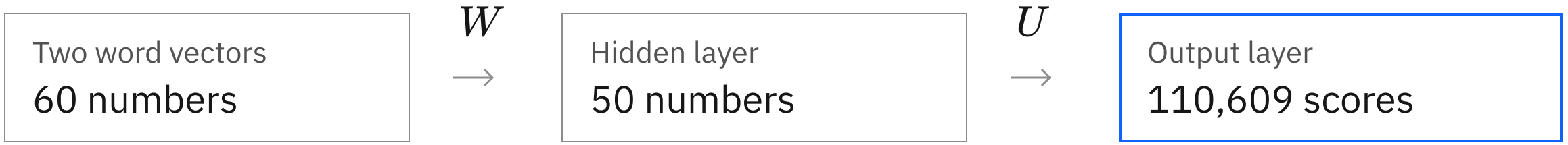
	N-gram counts	Neural language model
Numbers to store	1.4×10^{15} cells for trigrams	$V \times m$ in E, plus W, b and U
Unseen history	add-one: 1/7 for each next word	close to its prediction after film star
Similar words	each word counted on its own	can give them nearby vectors
How it is made	counted from the training texts	learned by gradient descent
Window	fixed: the last $N - 1$ words	fixed: the last $N - 1$ words

Both models see only a fixed number of previous words. Part 3 removes this limit.

Part 2: Word2vec

Word2vec learns word vectors from a simpler task than predicting the next word.

Where does the work go?



Hidden layer | $60 \times 50 = 3{,}000$

Output layer $50 \times 110{,}609 = 5{,}530{,}450$

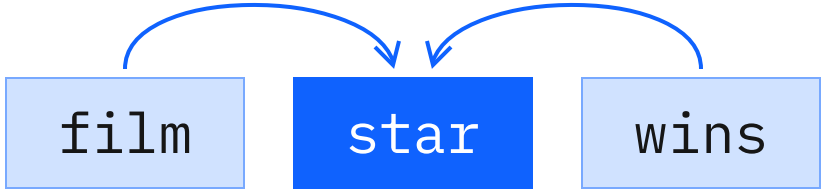
$$\hat{y}_{\text{wins}} = \frac{\exp(z_{\text{wins}})}{\sum_j \exp(z_j)}$$

Problem: almost all of the work is in the output layer, and this work grows with the size of the vocabulary.

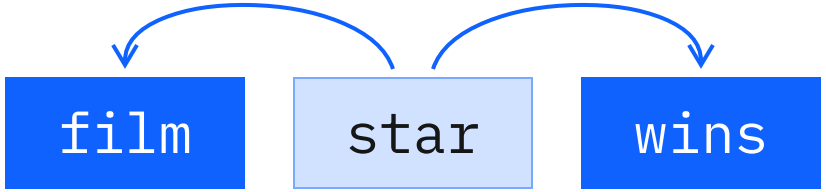
Do we need all this work if we want only the word vectors?

Two simpler models

CBOW



Skip-gram



$$\text{score} = c \cdot w$$

w a word's vector as the middle word

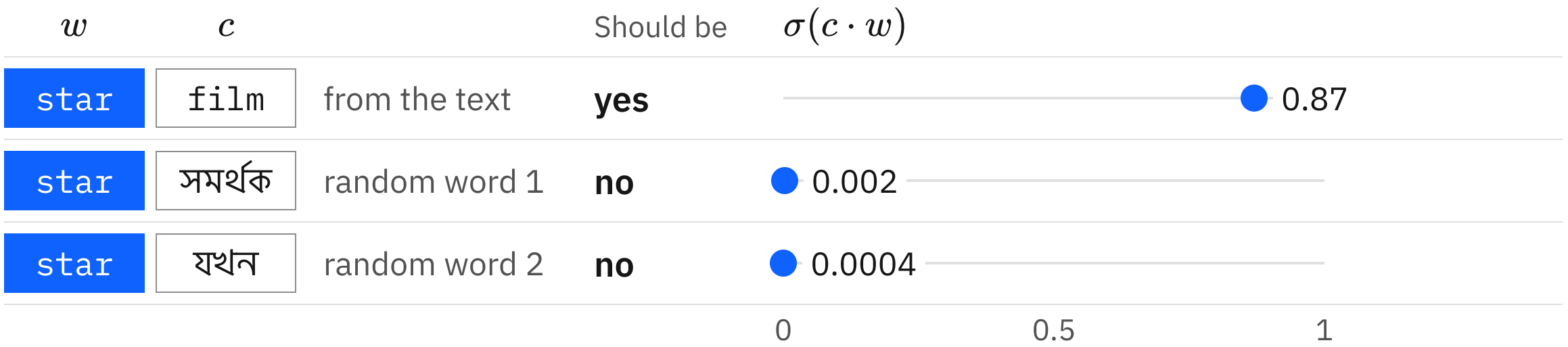
c its second vector, as a neighbor

$30 \times 110{,}608 = 3{,}318{,}240$ multiplications for each prediction, with softmax over every word

Problem: 3,318,240 is less than 5,530,450, but it still grows with the size of the vocabulary.

Can we ask the model a smaller question?

A yes-or-no question



$$L = -\log \sigma(c_{\text{film}} \cdot w_{\text{star}}) - \log\left(1 - \sigma(c_1 \cdot w_{\text{star}})\right) - \log\left(1 - \sigma(c_2 \cdot w_{\text{star}})\right)$$

$3 \times 30 = 90$ multiplications, not 3,318,240

Solved: the work of learning the vectors no longer depends on the size of the vocabulary.

What do the vectors show?

film		**presidential**		**নির্বাচন**		**ক্রিকেট**	
festival	0.97	primary	0.97	নির্বাচনের	0.94	বিসিবি	0.94
movie	0.97	republican	0.96	ইসি	0.93	পিসিবি	0.93
fest	0.96	candidate	0.96	কমিশনকে	0.92	বিসিসিআই	0.91
disney	0.95	candidates	0.95	তফসিল	0.92	পাপন	0.90
avengers	0.95	debate	0.95	ব্যালট	0.91	আইসিসি	0.89

`king − man + woman`

Far more text
nearest other word: queen

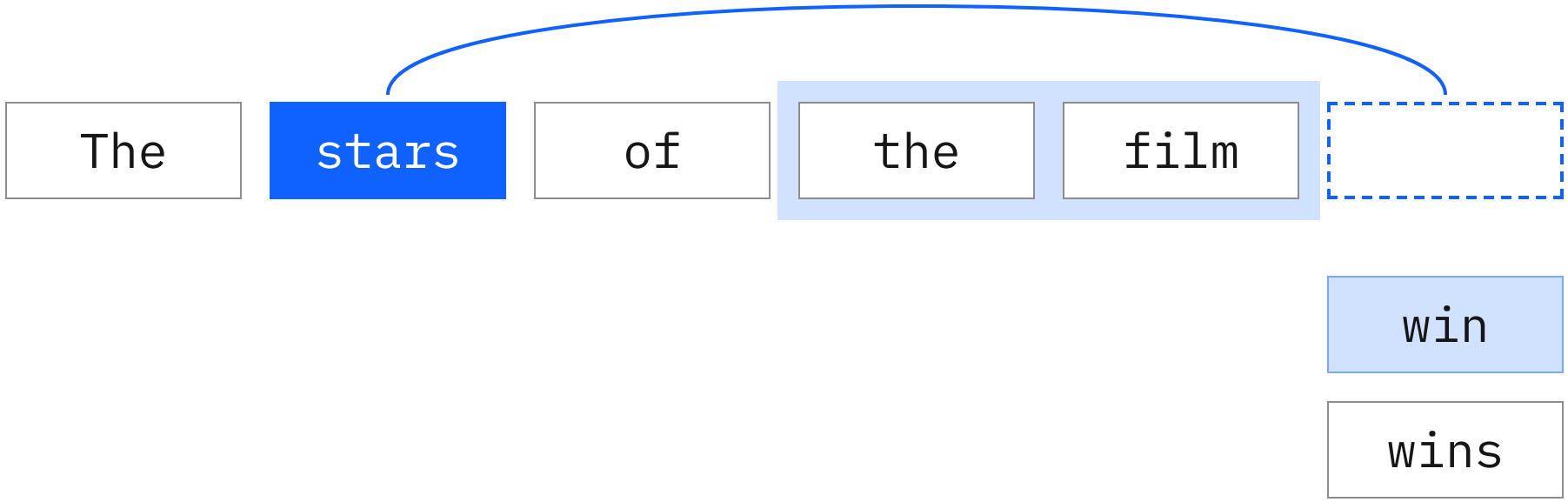
C1, 1.8 million words
queen is not among the five nearest

Word2vec does not predict the next word, but for years other models used its vectors w as their table E.

Part 3: Recurrent networks

A recurrent network reads a sentence one word at a time and passes its hidden layer on to the next word.

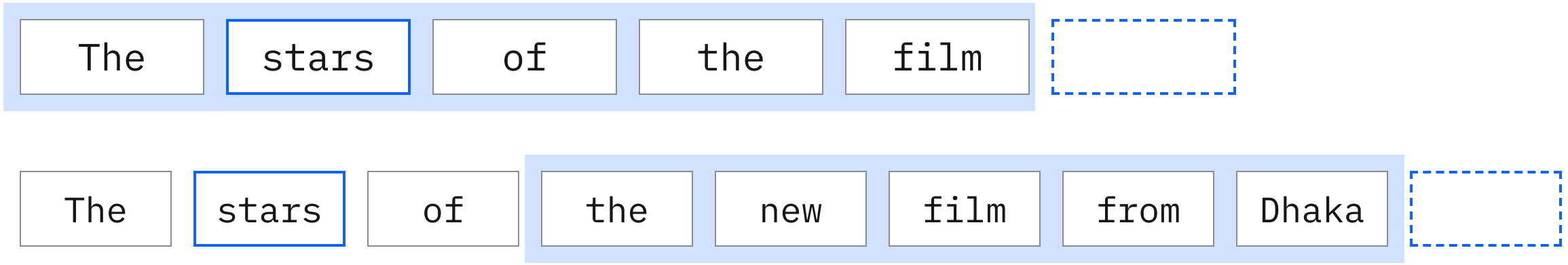
How far back can the model see?



Problem: the model cannot see stars, which lies outside its window of two words.

Would a wider window help?

Why not a wider window?



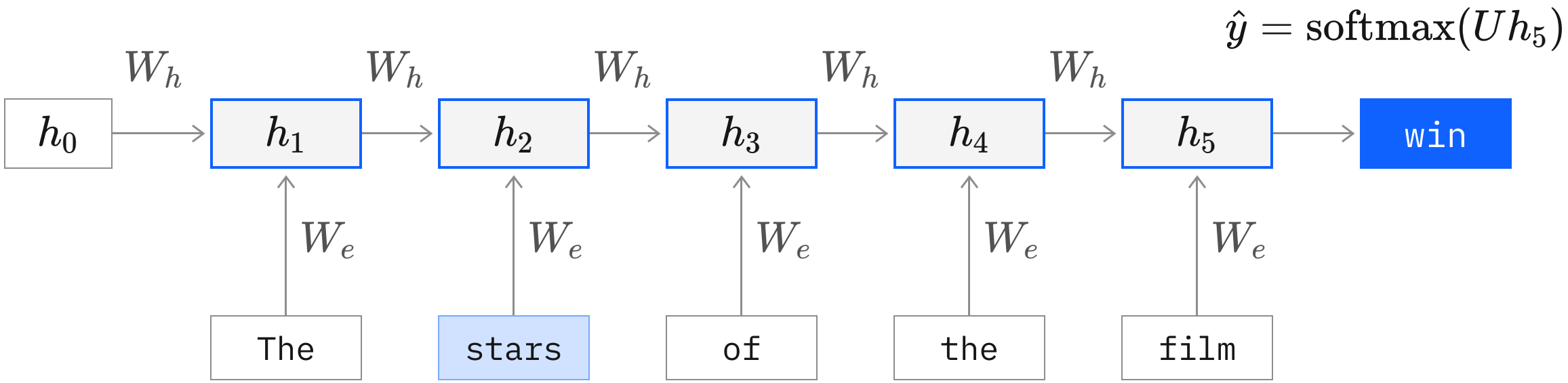
Window of 2 words	$2 \times 30 \times 50 = 3{,}000$	weights in W
Window of 5 words	$5 \times 30 \times 50 = 7{,}500$	weights in W

Problem: a word that matters can lie outside any fixed window, and each place needs its own weights.

Can one set of weights read a history of any length?

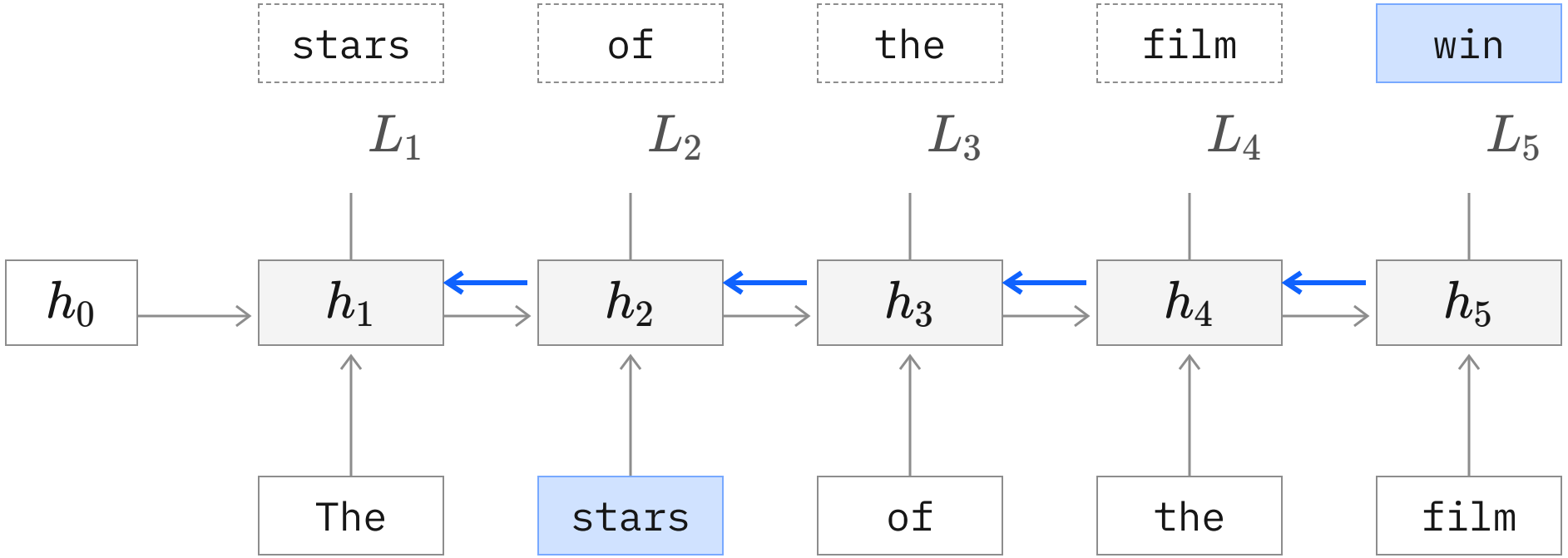
A simple recurrent network

$$h_t = \sigma(W_h\, h_{t-1} + W_e\, e_t + b)$$



Solved: the state depends on every earlier word, and the number of weights does not grow with the length of the history.

Training through time



$$L = (L_1 + L_2 + L_3 + L_4 + L_5)/5$$

The gradient from win must go back three time steps, from h_5 to h_2, to reach stars. What happens to it on the way?

What happens on the way back?

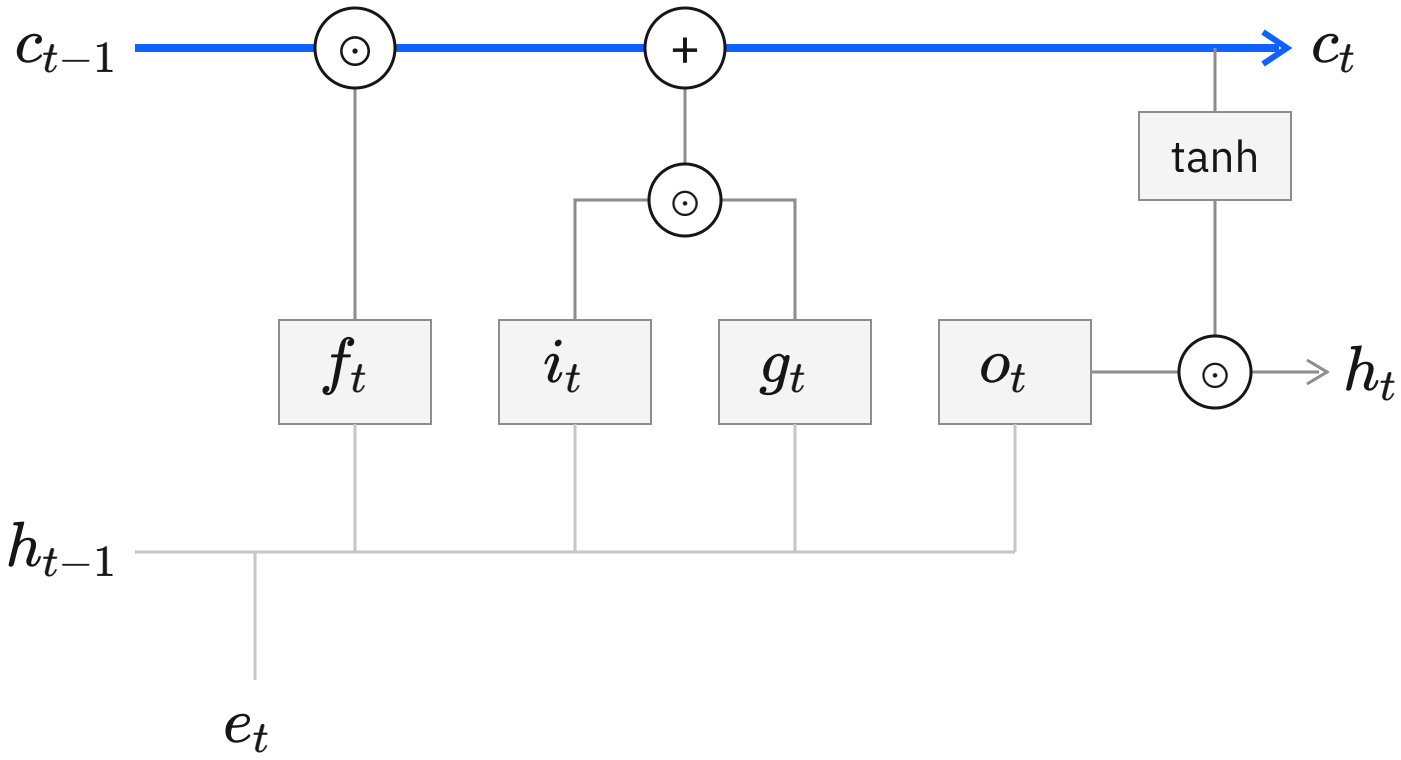
Steps back	1	2	3	4	5	6	7	8
Factor 2	2	4	8	16	32	64	128	**256**
Factor 0.5	0.5	0.25	0.125	0.063	0.031	0.016	0.008	**0.004**

Clipping $\|[6,\ 8]\| = 10 > \tau = 5: \quad [6,\ 8] \times \frac{5}{10} = [3,\ 4]$

Problem: the gradient vanishes, so the network does not learn to keep information about a word far back.

Can a network carry information over many words without shrinking it?

The LSTM



$$[1,\ 0] \odot [4,\ 6] = [4,\ 0]$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

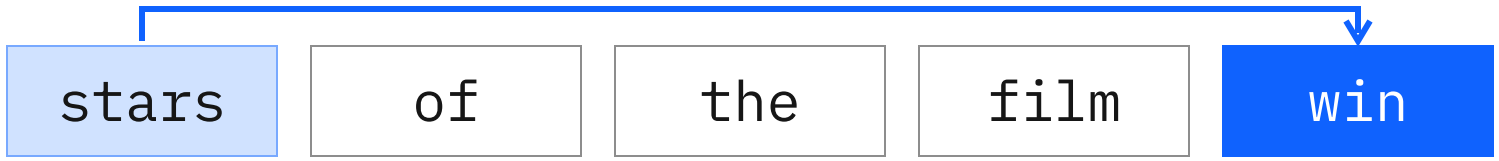
$$h_t = o_t \odot \tanh(c_t)$$

What happens to the gradient on its way back along the cell?

What happens along the cell?

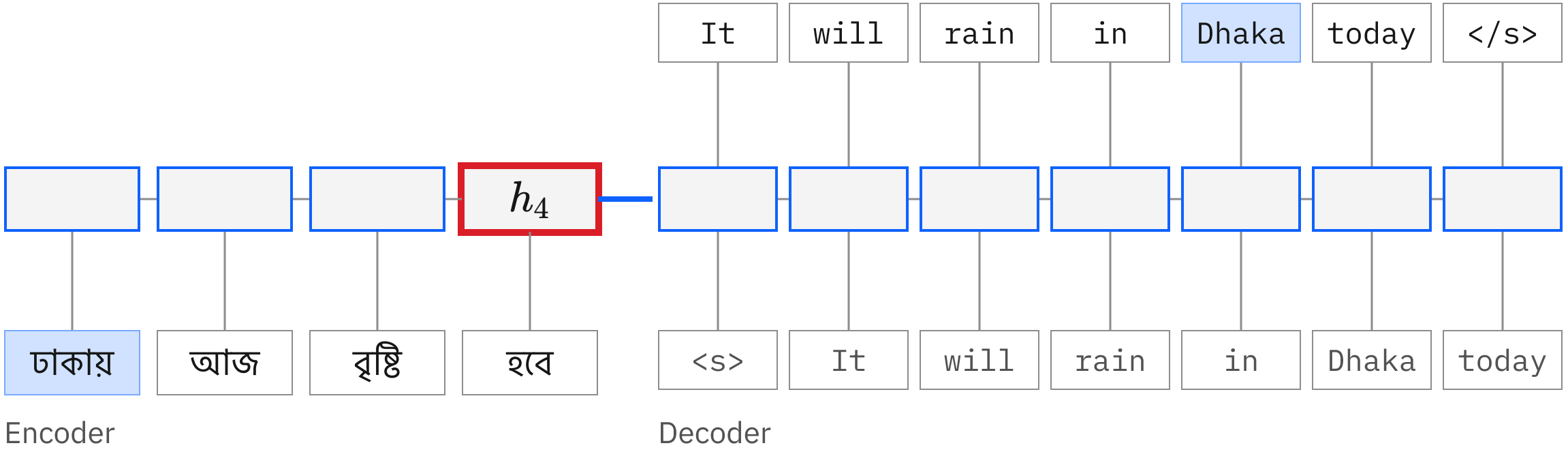
Steps back	1	2	3	4	5	6	7	8
Forget gate 0.99	0.99	0.98	0.97	0.96	0.95	0.94	0.93	**0.92**
Simple network 0.5	0.5	0.25	0.125	0.063	0.031	0.016	0.008	**0.004**

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$



Solved: with the forget gate near 1, the cell keeps information over many words.

Translation with two networks



Problem: the decoder sees the whole Bangla sentence only through one vector, h_4, however long the sentence is.

What else is hard for a recurrent network?

What is still hard?

One word at a time

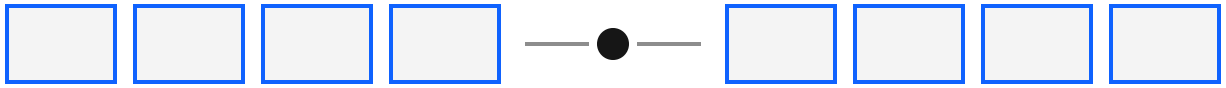


Words far back



$$0.99^{200} \approx 0.13$$

One vector



When the decoder writes Dhaka, can it look directly at ঢাকায়?

What we learned

Limits of counting	Counts need a cell for every n-gram, and most n-grams of new text were never seen.	1.4×10^{15} trigram cells
Words used alike	Words that fit the same blanks are used alike, but one-hot vectors put every pair at cosine 0.	$\cos(\text{film}, \text{movie}) = 0$
The neural language model	Each word gets a vector, and the next word is predicted from the vectors. Both are learned from the loss.	$h = \sigma(We + b), \quad \hat{y} = \text{softmax}(Uh)$
What it learns	Film and movie get nearby vectors. On C1, the dev loss per word falls below every n-gram model.	Dev loss: 7.29 against 8.54

What we learned

Word2vec	Word vectors are learned from yes-or-no questions about neighbors in the text, without a score for every word.	90 multiplications against 3,318,240
Simple recurrent network	The state depends on every earlier word, and the same weights are used at every word.	$h_t = \sigma(W_h h_{t-1} + W_e e_t + b)$
LSTM	A memory cell, updated by addition, keeps information over many words while the forget gate stays near 1.	$c_t = f_t \odot c_{t-1} + i_t \odot g_t$

References

Textbook

Jurafsky, D. and Martin, J. H. (2026). *Speech and Language Processing*, 3rd ed. draft, August 2026. Chapter 6, section 6.5, pp. 163–168 (the feedforward neural language model, eq. 6.24, fig. 6.14). Chapter 5, section 5.5, pp. 136–142 (word2vec; eqs. 5.15, 5.16 and 5.21); section 5.7, pp. 143–145 (analogies). Chapter 14, sections 14.1 and 14.2, pp. 302–309 (eqs. 14.1–14.6, pp. 304–307); section 14.5, pp. 314–318 (the LSTM, eqs. 14.20–14.27, p. 316); sections 14.7 and 14.8, pp. 319–323 (the encoder-decoder model and its bottleneck).

Papers

Bengio, Y., Ducharme, R., Vincent, P. and Jauvin, C. (2003). *A neural probabilistic language model.* Journal of Machine Learning Research 3, 1137–1155.

Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013). *Efficient estimation of word representations in vector space.* ICLR 2013 workshop, arXiv:1301.3781. Mikolov, T., Sutskever, I., Chen, K., Corrado, G. and Dean, J. (2013). *Distributed representations of words and phrases and their compositionality.* NIPS 26, 3111–3119. Mikolov, T., Yih, W. and Zweig, G. (2013). *Linguistic regularities in continuous space word representations.* NAACL-HLT 2013, 746–751.

Elman, J. L. (1990). *Finding structure in time.* Cognitive Science 14(2), 179–211. Hochreiter, S. and Schmidhuber, J. (1997). *Long short-term memory.* Neural Computation 9(8), 1735–1780. Gers, F. A., Schmidhuber, J. and Cummins, F. (2000). *Learning to forget: continual prediction with LSTM.* Neural Computation 12(10), 2451–2471.

Pascanu, R., Mikolov, T. and Bengio, Y. (2013). *On the difficulty of training recurrent neural networks.* ICML 2013, PMLR 28(3), 1310–1318. Sutskever, I., Vinyals, O. and Le, Q. V. (2014). *Sequence to sequence learning with neural networks.* NIPS 27, 3104–3112.

References

Data and code

The scripts c04_build.py (the toy neural language model on the ten toy articles of class 2 and seven more), c04_neural_c1.py (one neural language model per topic on C1) and c04_word2vec_c1.py (skip-gram with negative sampling on C1), on the C1 training set (40,000 texts) and dev set (4,000 texts).

History

Markov (1913), on vowels and consonants in *Eugene Onegin*. English translation: Science in Context 19(4), 591–600, 2006. Shannon (1948). Bell System Technical Journal 27(3), 379–423. Spärck Jones (1972). Journal of Documentation 28(1), 11–21. Bahl et al. (1978). ICASSP 1978, 422–424. Katz (1987). IEEE Transactions on Acoustics, Speech, and Signal Processing 35(3), 400–401. Rumelhart, Hinton and Williams (1986). Nature 323, 533–536. Robertson et al. (1994), BM25. Okapi at TREC-3. Bengio, Ducharme and Vincent (2000). NIPS 13, 932–938. Google blogs of 28 April 2006, 27 September 2016 and 25 April 2017. ACM Turing Award 2018; Nobel Prize in Physics 2024.

Photos

Yoshua Bengio: Xuthoria, 2025. CC BY-SA 4.0, via Wikimedia Commons, File:ICLR 2025 - Yoshua Bengio 03.jpg; cropped and made grayscale.

Geoffrey Hinton: Arthur Petron, 2024. CC BY-SA 4.0, via Wikimedia Commons, File:Geoffrey E. Hinton, 2024 Nobel Prize Laureate in Physics.jpg; cropped and made grayscale.

Yann LeCun: Jérémy Barande / Ecole polytechnique Université Paris-Saclay, 2018. CC BY-SA 2.0, via Wikimedia Commons, File:Yann LeCun (41208595340).jpg; cropped and made grayscale.

Thank you.